



Fuse Reference

Creation

FuRegisterClass()

Summary

The FuRegisterClass function is executed when Fusion first loads the Fuse tool or ScriptViewShader. The arguments to this function provide Fusion with the information needed to properly present the tool for use by the artist. Fusion must be restarted before edits made to this function will take effect.

The FuRegisterClass function is required for all Fuse tools and ScriptViewShaders, and generally appears as the first few lines of the Fuse script.

Usage

FuRegisterClass(*string* name, *enum* ClassType, *table* attributes)

Returns

This event function does not return a value.

Arguments

name(*string*, required)

The name argument is a unique identifier that is used to identify the plugin to Fusion. It is also used as the base for the tool's default name. For example, the first instance a ScriptPlugin with the name 'Bob' would be added to the flow as Bob1.

NOTE The name should use only characters between A-Z, 0-9 and the underscore, and should not start with a number. For example, a Fuse named sample-tool would appear to work, but would actually create compositions which can not be reopened.

ClassType (enum, required)

The ClassType is a predefined variable which identifies the type of Fuse for Fusion. Some valid values for the ClassType are :

- CT_Tool
- CT_Modifier
- CT_ViewLUTPlugin

attributes (table, required)

The attributes table defines all the remaining options needed to define a Fusion tool. There are a wide variety of possible attributes, and not all are required. The following table lists the most common attributes, and their expected values. A more comprehensive list can be found at FuRegisterClass Attributes.

REGS_Category

Required. A string value which sets the category a tool will appear in. For example, REGS_Category = "Script" will place the tool in the Scripts category of the tool menu. If the category does not exist, it will be created to hold the tool. Nested Categories can be defined using a \ character as a separator. For example, REGS_Category = "Script\\Color" will create a Color category under the Script category of the tool menu. Remember to use \\ instead of \ in a regular string, as \ is considered an escape character unless the [[]] syntax is used.

REGS_OpIconString

Required. A string value that defines the abbreviation of the tools name. This is used in the toolbar menu and by the bins.

REGS_OpDescription

Required. A short description of the tool, used in the various parts of the Fusion interface.

REGS_Name

Optional. Only needed if the ViewShader's displayed name is different to its unique ID.

Examples

The following shows the FuRegisterClass function used to create a Tool called Ex2_Controls, In the Tools menu Fuses\\Examples

```
FuRegisterClass("ExampleControls", CT_Tool, {
    REGS_Name = "Ex2_Controls",
    REGS_Category = "Fuses\\Examples",
    REGS_OpIconString = "E2C",
    REGS_OpDescription = "Example, showing the various Controls in Fusion",
    REGS_HelpTopic = "Example Location of Help", -- This can be a URL
    REGS_URL = "www.blackmagicdesign.com",
    REGS_IconID = "Icons.Tools.Icons.Example", -- This can be inline as
    an array
    REG_OpNoMask = false, -- Mask Input shows and will mask the
    output image
    REG_NoBlendCtrls = true, -- This will allow whether this tool
    can Blend
    REG_NoObjMatCtrls = true, --Set this to allow masking from the
    Object mattes
    REG_NoMotionBlurCtrls = true, -- Set whether Motion blur will work
    with this tool
    REG_NoBlendCtrls = false, -- This will allow whether this tool can
    Blend
    REG_Fuse_NoEdit = false, -- To not allow editing of the Fuse set
    to true
    REG_Fuse_NoReload = false, -- To no allow reloading of the Fuse
    set to true
    REG_Version = 1,
})
```

Create

Summary

The Create event function is executed whenever the Fuse tool is added to the composition. It should contain all of the information required to draw the tools inputs and outputs, and to display the tools controls in the control window.

The Create function does not require or use any arguments and does not return a value.

While all Fuse tools MUST provide a Create event function, that function can be empty.

Usage

Create()

Arguments

None

Examples

All Fuse tools have a Create function. See the [Example_Fuses](#) page.

Process

Summary

The Process function is called whenever the Fuse Plugin tool needs to do some work by processing the image. This can occur during the final render, or during an interactive render. The Request object passed to the Process function as its only argument contains information about the current render settings, including current time, proxy and motion blur settings.

The process function does not return a value.

All Fuse tools require a Process event function.

Usage

Process(*object* Request)

Arguments

Request (*object*, required) The request object is automatically passed to the Process function when Fusion invokes the Process event. This object contains all the relevant information about the current render request, including proxy and motion blur settings, and the current state of all tools.

In Fuse tools released by eyeon, the Request object is usually assigned to the variable 'req'.

Examples

See [Fuse Examples](#) for further information.

NotifyChanged

Summary

The NotifyChanged event function is executed any time a control is changed on a tool. It executes before the Process event function. Typically the NotifyChanged event is used to adjust the values of controls before they are locked for rendering.

For example, the NotifyChanged function may check to see if a Blur tools Lock X/Y Strength checkbox has been selected. If it has, it could take care of hiding or disabling the Y strength slider, and setting the Y strength to match the X strength.

Usage

NotifyChanged(object input, object parameter, number time)

Arguments

input (*Input* object)

The Input object whose control has changed

parameter (*Parameter* object)

The new Parameter object produced by the control (may be nil, datatype depends on the Input)

time (number)

The current frame number, as shown on the comp's timeline.

Example

```
function NotifyChanged(inp, param, time)
  if inp == InNotify then -- If Notify Change check box is changed,
                           then rename the Control names and Un/
                           Hide sliders
    local locked = (param.Value > 0.5)
    if locked then
      InSliderH:SetAttrs({ LINKS_Name = "Slider Hide/Show" })
      InSliderH:SetAttrs({ IC_Visible = true })
      InNotify:SetAttrs({ LINKS_Name = "Notify Change
                          Showing" })
    else
      InSliderH:SetAttrs({ LINKS_Name = "Slider Hide/Show
                          Hidden" })
      InSliderH:SetAttrs({ IC_Visible = false })
      InNotify:SetAttrs({ LINKS_Name = "Notify Change Hidden"
    })
  end
end
end
```

OnAddToFlow

Summary

The OnAddToFlow event function is executed when the tool is added to the flow. It executes before the Process event function. Typically the OnAddToFlow event is used to set the values of controls.

Usage

OnAddToFlow()

Arguments

None

Example

This Example shows setting colors in a Color Gradient control

```
-- OnAddToFlow can be used to set parameters and other processing
functions when the Tool is add to a comp
function OnAddToFlow()
    local grad = Gradient()
    if InNewTool:GetSource(0).Value >= 0.5 then
        -- There is no default attribute for gradients. It's always
        black to white.
        -- To have a different gradient for new tools, we'll set one
        up here but
        -- only once. The "NewTool" flag is cleared immediately
        afterwards so we
        -- don't overwrite gradients when the comp is reopened at a
        later time.
        grad:AddColor(0.0, Pixel({R = 1.0, G = 0, B = 0, A = 1}))
        grad:AddColor(0.5, Pixel({R = 0, G = 1.0, B = 0, A = 1}))
        grad:AddColor(1.0, Pixel({R = 0, G = 0, B = 1.0, A = 1}))
        InGradient:SetSource(grad, 0, 0)
        InNewTool:SetSource(Number(0.0), 0, 0)
    end
end
```

Input

There are a number of Inputs to a Fuse, Images, Numbers and 2D Points with XY values.

GetValue

Gets the current value (Parameter) of this Input from a Request

Summary

The GetValue function is used to retrieve the current values of a control from the current render request. The Request object is the only argument this function will accept. GetValue returns either a value or object which represents the current properties of an Input.

Usage

object:GetValue(*object* Request)

request (*required*, object)

The Request object is always passed to the Process event function as an argument. See the pages for the Request object and Process event function for more information.

GetValue() returns an object (a subclass of the Parameter class). To get the value as a variable that LUA can process further or compare to other data types, use the corresponding member of the returned object:

- Point .X and .Y returns float values of the x and y coordinates
- Text .Value returns a string variable
- FuID .Value returns a string variable containing the ID

GetSource

Gets the value of the Input at any given time

Summary

The GetSource function is used to return the value of an Input at a time different from the current time. For example, GetSource could be used to produce images from the frames before and after the current frame, or to average the values of a Blur slider across 10 frames.

Usage

Input:GetSource(*number* frame)

frame (*number*, required)

A numeric value representing which frame will be read to produce the value.

```
function Process(req)
    local img = InImage:GetSource(req.Time + 5) --Gets the image 5
    frames from now
    OutImage:Set(req, img)
end
```

SetSource

Sets the Input to a given value

Summary

The SetSource function is used to set an Input to a specified value.

Usage

Input:SetSource(*various value*, *number* time)

value (*object*, required)

The value which should be assigned to the input. The type will vary according to the inputs LINKID_DataType attribute. The value should be one of Fusion's datatypes, like Number, Text, or Point. This means a call like Input:SetSource("some text", 0) would fail, but Input:SetSource(Text("some text"), 0) would work.

time (*number*, required)

The frame at which to set the input value. If an input is animated, this will ensure the value sets the appropriate keyframe.

```
function Create()
    InRed = self:AddInput("Red", "Red", {
        INPID_InputControl = "SliderControl",
        INP_Default = 1.0,
        INP_DoNotifyChanged = true,
        INP_External = false
    })
    ~~~~~
function Process(req)
    InRed:SetSource(Number(5.0),0)--This will set the slider to 5.0
```

GetAttr

Gets the value of a specified attribute

Summary

The GetAttr() function is used to retrieve the value of a specific attribute of this object.

Usage

```
result = Input:GetAttr(string attribute)
```

attribute (*string*, required)

The name of the attribute to query.

result (various)

The value of the specified attribute. The type will vary according to the attribute.

SetAttrs

Sets a table of tag attributes into the Input

Attributes

Name	Type : Description
INPID_InputControl	string : The ID of the type of tool window control used by the input.
INPID_PreviewControl	string : The ID of the type of display view control used by the input.
INPID_AddModifier	string : A tool of this type ID should be automatically created and connected to this input, at creation time.
INPID_DefaultID	string : Inputs of datatype "FuID" should use this ID as their default value.
INPS_DefaultText	string : Inputs of datatype "Text" should use this string as their default.
INPS_StatusText	string : The text shown on the status bar on mouse hover.
INP_External	boolean : Whether this input can be animated or connected to a tool or modifier.
INP_Active	boolean : This input's value is used in rendering.
INP_Required	boolean : The tool's result requires a valid Parameter from this input.
INP_Connected	boolean : The input is connected to another tool's Output.
INP_Priority	integer : Used to determine the order in which the tool's inputs are fetched.
INP_Disabled	boolean : The input will not accept new values.
INP_DoNotifyChanged	boolean : The tool is notified of changes to the value of the input.
INP_Integer	boolean : The input rounds all numbers to the nearest integer.
INP_NumSlots	integer : The number of values from different times that this input can fetch at once.

Name	Type : Description
INP_ForceNotify	boolean : The tool is notified whenever a new parameter arrives, even if it is the same value.
INP_InitialNotify	boolean : The tool is notified at creation time of the initial value of the input.
INP_Passive	boolean : The value of this input will not affect the rendered result, and does not create an Undo event if changed.
INP_InteractivePassive	boolean : The value of this input will not affect the rendered result, but it can be Undone if changed.
INP_AcceptTransform	boolean : This input will also accept TransformMatrix parameters.
INP_AcceptsMasks	boolean : This input will also accept Mask images.
INP_AcceptsGLImages	boolean : This input will also accept Images with attached OpenGL textures.
INP_MinAllowed	number : Minimum allowed value - any numbers lower than this value are clipped.
INP_MaxAllowed	number : Maximum allowed value - any numbers higher than this value are clipped.
INP_MinScale	number : The lowest value that the input's control will normally display.
INP_MaxScale	number : The highest value that the input's control will normally display.
INP_Default	string : Inputs of datatype "Number" should default to this value.
INP_DefaultX	string : Inputs of datatype "Point" should use this as their default X value.
INP_DefaultY	string : Inputs of datatype "Point" should use this as their default Y value.

Tips for Attributes

- **INP_InitialNotify** defaults to true and is only done if **INP_DoNotifyChanged** is also true.
- There are no attributes called **INPS_Name** or **INPS_ID** like in the eyeonscript API. Use **LINKS_Name** and **LINKID_ID** instead, to get the name or script ID of an input.
- **INP_SendRequest**, which isn't listed here, defaults to true and controls whether the input will be requested before `Process()` is called. If you set this to false on image inputs, for example, you can prevent Fusion from rendering connected branches before you have decided if you need them or not. In this case, you cannot use `Input:GetValue(req)` to get the input's value. Use `Input:GetSource(time)` instead. Also, if you use `INP_SendRequest = false` on the main image input, the Fuse will fail if you don't take care of `PreCalcProcess()` yourself (or set `INP_Required = false`).
- **INP_AcceptsDoD**, Set to false to turn off RoI for an image input. Fusion will in this case always request the full image window just as if the Fuse was a non-DoD tool.